

Capitolo 3 - Processi

Status Done

Informalmente, un processo è un programma in esecuzione. Lo stato dell'attività corrente di un processo è rappresentato:

- dal valore del contatore di programma;
- dal contenuto dei registri del processore.

La struttura di un processo in memoria è suddivisa in più sezioni:

- **sezione di testo**: contenente il codice eseguibile;
- **sezione dati**: contenente le variabili globali;
- **heap**: memoria allocata dinamicamente durante l'esecuzione del programma;
- **stack**: memoria temporaneamente utilizzata durante le chiamate di funzioni.

Le dimensioni delle sezioni di testo e dati sono fisse, mentre stack e heap possono ridursi e crescere dinamicamente durante l'esecuzione.

Ogni volta che si chiama una funzione, un record di attivazione (*activation record*) contenente i suoi parametri, le variabili locali, e l'indirizzo di ritorno viene inserito nello stack; quando la funzione restituisce il controllo al chiamante, il record di attivazione viene rimosso dallo stack.

L'heap cresce quando viene allocata memoria dinamicamente e si riduce quando la memoria viene restituita al sistema.

Le sezioni stack e heap crescono l'una verso l'altra è il SO a dover garantire che non si sovrappongono.

Un programma di per sé non è un processo, ma un'entità passiva. Un processo è un'entità attiva, con un contatore di programma che specifica qual è l'istruzione successiva da eseguire e un insieme di risorse associate. Un programma diventa un processo quando il file eseguibile viene caricato in memoria.

Due processi sono associabili allo stesso programma, ma devono essere considerate due sequenze d'esecuzione distinte.

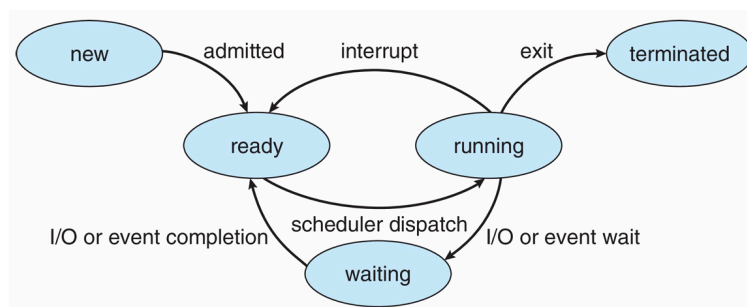
Un processo può essere un ambiente d'esecuzione per altro codice, come ad esempio un programma java che viene eseguito all'interno della jvm. La jvm è in esecuzione come un processo che interpreta il codice java caricato e intraprende azioni per conto di quel codice.

Stato del processo

Un processo durante l'esecuzione è soggetto a cambiamenti del suo stato:

- **Nuovo**: si crea il processo.
- **Esecuzione (running)**: le sue istruzioni vengono eseguite.
- **Attesa (waiting)**: il processo attende che si verifichi qualche evento.
- **Pronto (ready)**: il processo attende di essere assegnato a un'unità d'elaborazione.
- **Terminato**: il processo ha terminato l'esecuzione.

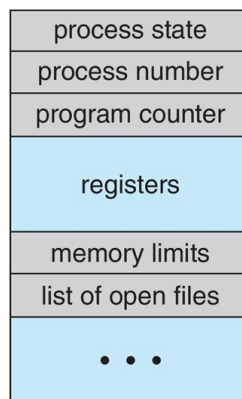
In ciascuna unità d'elaborazione può essere in *esecuzione* solo un processo per volta, sebbene molti processi possano essere *pronti* o nello stato di *attesa*.



Blocco di controllo del processo

Ogni processo è rappresentato nel SO da un **blocco di controllo** (*process control block*). Esso contiene informazioni connesse a uno specifico processo:

- **Stato del processo:** *new, ready, running, waiting, terminated, ecc.*
- **Contatore di programma:** contiene l'indirizzo della successiva istruzione da eseguire.
- **Registri della cpu:** accumulatori, registri indice, puntatori alla cima dello stack (*stack pointer*), registri d'uso generale e registri contenenti i codici di condizione (*condition codes*). Quando si verifica un'interruzione della cpu si devono salvare tutte queste informazioni insieme con il contatore di programma, per poter riprendere l'esecuzione del processo in un momento successivo.
- **Informazioni sullo scheduling di cpu:** le priorità del processo, i puntatori alle code di scheduling e altri parametri di scheduling.
- **Informazioni sulla gestione della memoria:** il valore dei registri di base e di limite, le tabelle delle pagine o le tabelle dei segmenti.
- **Informazioni di accounting:** quota di uso della cpu, tempo di utilizzo di essa, limiti di tempo, numero dei processi, ecc.
- **Informazioni sullo stato dell'I/O:** lista dei dispositivi I/O assegnati a un determinato processo, elenco dei file aperti, ecc.



Thread

Il modello dei processi illustrato fin qui sottintende che un processo è un programma che si esegue seguendo un unico percorso d'esecuzione, detto *thread*. Nella maggior parte dei sistemi operativi moderni si è esteso il concetto di processo introducendo la possibilità d'avere più percorsi d'esecuzione, in modo da permettere che un processo possa svolgere più di un compito alla volta.

Scheduling dei processi

L'obiettivo della multiprogrammazione è avere sempre un processo in esecuzione per poter massimizzare l'utilizzo della cpu.

L'obiettivo del time-sharing è di commutare l'uso della cpu tra i vari processi così frequentemente che gli utenti possano interagire con ciascun programma mentre è in esecuzione.

Per far sì che tutto ciò sia possibile, lo scheduler dei processi seleziona un processo da eseguire dall'insieme di quelli disponibili.

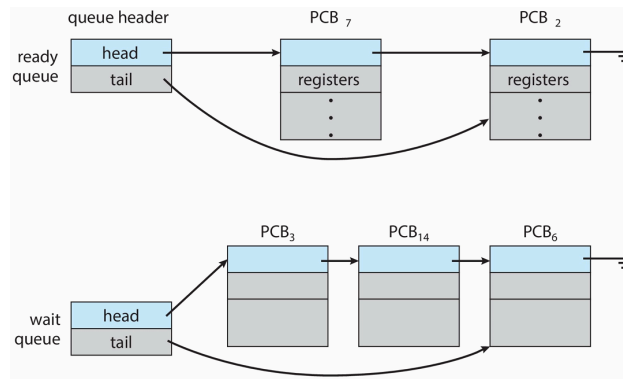
Ogni core della cpu può eseguire un processo alla volta, se vi sono più processi che core, i processi in eccesso dovranno attendere fino a quando un core diventa libero. Il numero di processi in memoria in un dato istante è noto come il **grado di multiprogrammazione**.

Il bilanciamento tra gli obiettivi della multiprogrammazione e la condivisione del tempo richiede di tener conto anche del comportamento complessivo di un processo:

- processo con prevalenza di I/O (*I/O bound*): impiega la maggior parte del tempo nell'esecuzione di operazioni I/O;
- processo con prevalenza d'elaborazione (*cpu bound*): richiede operazioni di I/O con poca frequenza e impiega la maggior parte del proprio tempo nelle elaborazioni.

Code di scheduling

Nel sistema, ogni processo è inserito in una coda di processi pronti e in attesa di essere eseguiti (*ready queue*). Generalmente, è memorizzata come una lista concatenata: un'intestazione della coda dei processi pronti contiene i puntatori al primo pcb della lista, e ciascun pcb comprende un campo puntatore che indica il successivo processo contenuto nella coda.

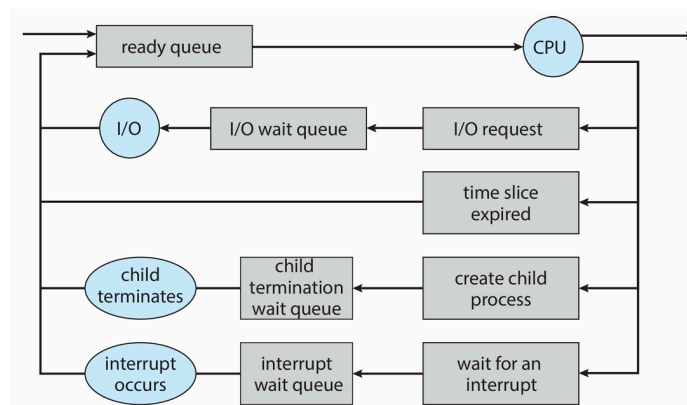


I processi in attesa di un determinato evento vengono collocati in una coda di attesa, o *wait queue*.

Una comune rappresentazione dello scheduling dei processi è data da un diagramma di accodamento.

Un nuovo processo si colloca inizialmente nella *ready queue*, dove attende finché non è selezionato per essere eseguito. Una volta che il processo è assegnato alla CPU ed è nella fase di esecuzione, si può verificare uno dei seguenti eventi:

- Il processo può emettere una richiesta di I/O e quindi essere inserito in una coda di I/O.
- Il processo può creare un nuovo processo figlio e attenderne la terminazione.
- Il processo può essere rimosso forzatamente dalla CPU a causa di un'interruzione ed essere reinserito nella coda dei processi pronti.



Cambio di contesto

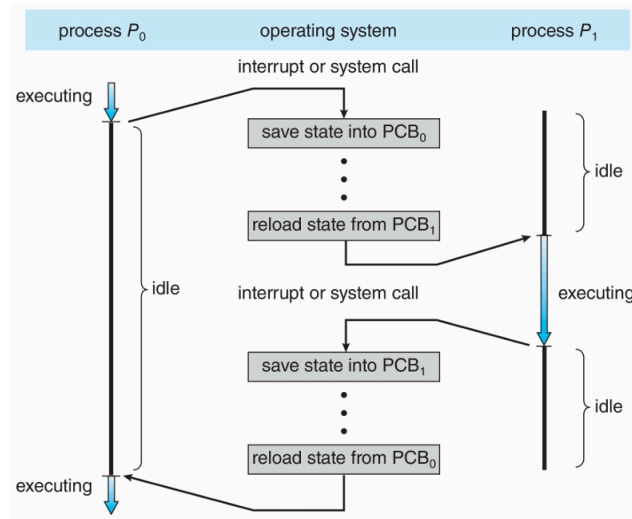
In presenza di una interruzione, il sistema deve salvare il contesto del processo corrente per poterlo ripristinare quando il processo potrà ritornare in esecuzione. Il contesto si trova all'interno del pcb e comprende:

- i valori dei registri di cpu,
- lo stato del processo,
- informazioni relative alla gestione della memoria.

⇒ Si esegue un salvataggio dello stato corrente della cpu in modo da poter riprendere l'elaborazione dal punto in cui era stata interrotta.

Il passaggio della cpu a un nuovo processo implica il salvataggio dello stato del processo attuale e il ripristino dello stato del nuovo processo ⇒ **Context Switch**.

Il sistema salva il contesto del processo uscente nel suo pcb e carica il contesto del nuovo processo, salvato in precedenza. Il cambio di contesto è puro overhead, in quanto vengono eseguite operazioni volte solo alla gestione dei processi e non alla computazione. Il tempo necessario varia da sistema a sistema (dipende dalla velocità della memoria, dal numero di registri da copiare e dall'esistenza di istruzioni macchina appropriate). La durata del cambio di contesto dipende dall'architettura.



Operazioni sui processi

Creazione di un processo

Durante la sua esecuzione, un processo può creare altri processi. Il processo creatore si chiama **genitore** (o **padre**) e il nuovo processo si chiama **figlio**. I nuovi processi possono a loro volta creare altri processi, formando un albero di processi. La maggior parte dei SO identifica un processo tramite un numero univoco, **pid** (*process identifier*), che può essere utilizzato per accedere ai vari attributi di un processo all'interno del kernel.

Quando un processo figlio viene creato, questo avrà bisogno di determinate risorse; può ottenerle:

- direttamente dal SO;
- può essere vincolato a un sottoinsieme delle risorse del processo genitore.

Il processo genitore può avere la necessità:

- di spartire le proprie risorse tra i processi figli;
- di condividerne solo alcune.

Limitando le risorse di un processo figlio a un sottoinsieme di risorse del processo genitore, si può evitare che un processo sovraccarichi il sistema creando troppi processi figli.

Quando un processo ne crea uno nuovo, ci sono due possibilità riguardo all'esecuzione:

1. il processo genitore continua l'esecuzione in modo concorrente con i propri processi figli;
2. il processo genitore attende che alcuni o tutti i suoi processi figli terminino.

Anche per quanto riguarda lo spazio d'indirizzi del nuovo processo ci sono due possibilità:

1. il processo figlio è un duplicato del processo genitore (ha gli stessi programmi e dati del genitore);
2. nel processo figlio si carica un nuovo programma.

Esempio Unix

Fase 1: Creazione del Processo Figlio (`fork()`):

- **Chiamata `fork()`**: Un processo esistente (il **Genitore**) invoca la chiamata di sistema `fork()`.
- **Duplicazione dello Spazio di Indirizzi**: Viene creato un nuovo processo (il **Figlio**) che è una **copia esatta** dello spazio di indirizzi del processo Genitore.
- **Identificazione del Processo**:
 - Il Genitore riceve il **PID (Process Identifier)** del processo Figlio (un valore intero **diverso da zero**).
 - Il Figlio riceve il valore **zero**.
- **Esecuzione Continuata**: Entrambi i processi, Genitore e Figlio, continuano l'esecuzione dall'istruzione immediatamente successiva alla chiamata `fork()`.

Fase 2: Sostituzione del Programma (`exec()`):

Dopo la `fork()`, solitamente uno dei due processi (spesso il Figlio) esegue il seguente passo:

1. **Chiamata `exec()`**: Il processo (Figlio) invoca una delle varianti della chiamata di sistema `exec()`.
2. **Caricamento del Nuovo Programma**: Il kernel **carica un file binario** specificato in memoria.
3. **Sovrascrittura**: Il nuovo file binario **sostituisce completamente** lo spazio di memoria, l'immagine e il codice del processo che ha chiamato `exec()`.
4. **Avvio dell'Esecuzione**: Il processo inizia l'esecuzione del **nuovo programma** dall'inizio.

Nota: La chiamata `exec()` non ritorna (non restituisce il controllo al programma chiamante) a meno che non si sia verificato un errore, poiché l'immagine del programma che l'ha invocata è stata cancellata e sostituita.

Fase 3: Sincronizzazione Opzionale del Genitore (`wait()`):

Mentre il Figlio esegue il nuovo programma, il Genitore può scegliere come procedere:

1. **Lavoro Concorrente**: Il Genitore può continuare a eseguire il proprio codice, operando **in parallelo** con il Figlio.
2. **Sospensione (`wait()`)**: Se il Genitore non ha altro da fare in attesa del risultato del Figlio, può invocare la chiamata di sistema `wait()`.
3. **Rimozione dalla Coda Pronti**: La chiamata `wait()` rimuove il Genitore dalla coda dei processi pronti e lo sospende.
4. **Ripresa**: Il Genitore rimane in attesa finché il processo Figlio **non termina** la sua esecuzione, dopodiché il Genitore viene risvegliato e riprende.

Terminazione di un processo

Un processo termina quando finisci l'esecuzione della sua ultima istruzione e inoltra la richiesta al SO di essere cancellato usando la chiamata di sistema `exit()`, il processo figlio riporta un'informazione di stato al processo padre, che la riceve attraverso la chiamata di sistema `wait()`. Tutte le risorse del processo sono liberate dal SO.

Un processo può terminare anche in altri casi, ad esempio un processo causa la terminazione di un altro attraverso un'opportuna chiamata di sistema; solitamente solo il genitore del processo da terminare (che conosce il pid del figlio) può invocare questo tipo di chiamata.

Un processo genitore può porre termine ad un processo figli perché:

- il processo figlio ha ecceduto nell'uso di alcune tra le risorse che gli sono state assegnate;
- il compito assegnato al processo figlio non è più richiesto;
- il processo genitore termina (condizioni normali o anomale) e il SO non consente a un processo figlio di continuare l'esecuzione in queste circostanze ⇒ **terminazione a cascata**.

Un processo che è terminato ma il cui genitore non ha ancora chiamato la `wait()`, è detto **zombie**. Una volta che la `wait` viene invocata il pid del processo zombie e la sua voce nella tabella dei processi vengono rilasciati.

Consideriamo ora che cosa accadrebbe se un genitore terminasse senza invocare la `wait()`, lasciando così orfani i suoi figli. Linux affronta questa situazione assegnando al processo `systemd` il ruolo di un nuovo genitore dei processi orfani. Il processo `init` invoca periodicamente `wait()`, consentendo in tal modo di raccogliere lo stato di uscita di qualsiasi processo orfano e rilasciando il suo pid e la relativa voce nella tabella dei processi. Nonostante la maggior parte dei sistemi Linux abbia sostituito `init` con `systemd`, quest'ultimo può assumere lo stesso ruolo, con la differenza che Linux permette anche ad altri processi di ereditare processi orfani e gestirne la terminazione.

Comunicazione tra processi

I processi eseguiti concorrentemente nel SO possono essere:

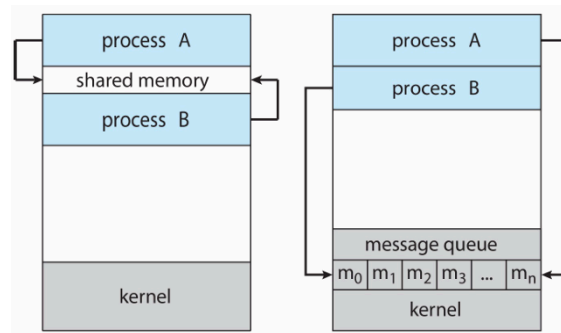
- **indipendenti**: se un processo non può influire su altri processi del sistema o subirne l'influsso;
- **cooperanti**: se un processo influenza o può essere influenzato da altri processi in esecuzione nel sistema.

La cooperazione tra processi può essere utile per:

- **condivisione di informazioni**;
- **velocizzazione del calcolo**, alcune attività d'elaborazione sono realizzabili più rapidamente se si suddividono in sottoattività eseguibili in parallelo;
- **modularità**, costruzione di un sistema modulare che suddivida le funzioni di sistema in processi o thread distinti.

I processi cooperanti necessitano di un meccanismo di comunicazione ⇒ IPC (*Interprocess Communication*). I modelli fondamentali sono:

- a **memoria condivisa**: si stabilisce una zona di memoria condivisa dai processi cooperanti;
- a **scambio di messaggi**: i processi cooperanti comunicano tramite messaggi.



IPC in sistemi a memoria condivisa

Questo tipo di comunicazione richiede che i processi comunicanti allochino una zona di memoria condivisa, solitamente residente nello spazio degli indirizzi del processo che la alloca. Solitamente il SO impedisce ad altri processi di accedere alla memoria riservata ad un altro processo, ai fini della comunicazione serve che due o più processi raggiungano un accordo per superare questo limite. Il tipo e la collocazione dei dati sono determinati dai processi e fuori dal controllo del SO. I processi non devono scrivere nella stessa locazione simultaneamente.

Il problema del produttore/consumatore è un paradigma utile per spiegare la cooperazione tra processi.

- Produttore: crea o genera informazioni.
- Consumatore: utilizza o elabora le informazioni prodotte.

Per l'esecuzione concorrente dei due processi, è essenziale un'area di memoria accessibile a entrambi, il **buffer**.

1. **Buffer condiviso**: un'area di memoria comune che il produttore riempie e il consumatore svuota.
2. **Sincronizzazione**: i processi devono essere sincronizzati per garantire che:
 - il consumatore non tenti di consumare un elemento non ancora prodotto;
 - (nel caso di buffer limitato) il prodotto non tenti di produrre un elemento quando il buffer è pieno.

Tipi di buffer

Tipo di Buffer	Caratteristiche	Problemi di Attesa
Illimitato	Non ha limiti pratici di dimensione.	Il Consumatore attende se il buffer è vuoto. Il Produttore non attende mai.
Limitato (Bounded)	Ha una dimensione fissa (<i>BUFFER_SIZE</i>).	Il Consumatore attende se il buffer è vuoto . Il Produttore attende se il buffer è pieno .

Implementazione del buffer limitato

Nella soluzione a memoria condivisa, il buffer limitato viene implementato tipicamente come un **array circolare** (o *buffer circolare*) con due variabili intere (puntatori logici) condivise:

- **in**: indica la successiva posizione libera dove il produttore inserirà il prossimo elemento.
- **out**: indica la prima posizione piena da cui il consumatore preleverà il prossimo elemento.

Codice del produttore:

```
item next_produced;
while (true) {
    /* 1. Produce un elemento in next_produced */

    // Attesa: loop vuoto finché il buffer non è pieno
    while (((in + 1) % BUFFER_SIZE) == out)
        ;

    // 2. Inserisce l'elemento nel buffer
    buffer[in] = next_produced;
```

```

    // 3. Aggiorna il puntatore 'in'
    in = (in + 1) % BUFFER_SIZE;
}

```

Codice del consumatore:

```

item next_consumed;
while (true) {

    // Attesa: loop vuoto finché il buffer non è vuoto
    while (in == out)
        ;

    // 1. Preleva l'elemento dal buffer
    next_consumed = buffer[out];

    // 2. Aggiorna il puntatore 'out'
    out = (out + 1) % BUFFER_SIZE;

    /* 3. Consuma l'elemento in next_consumed */
}

```

Questa soluzione ignora cosa succede se il produttore e il consumatore tentano di accedere o modificare le variabili condivise contemporaneamente.

Soluzione con buffer pieno (utilizzo di un contatore)

La soluzione vista limitava la capacità del buffer a $BUFFER_SIZE - 1$. Per permettere che tutti i buffer siano riempiti, si introduce una variabile intera condivisa `counter`.

- **Scopo:** tracciare il numero esatto di celle piene presenti nel buffer.
- **Implementazione:** inizialmente, il `counter` è impostato a 0.
- **Aggiornamento:** il produttore incrementa `counter` dopo aver prodotto un elemento; il consumatore decrementa `counter` dopo aver consumato un elemento.

Codice del produttore:

Il produttore deve attendere solo se il numero di elementi pieni (`counter`) è uguale alla dimensione massima del buffer (`BUFFER_SIZE`).

```

while (true) {
    /* produce un elemento in next_produced */

    // Attesa se il buffer è completamente pieno
    while (counter == BUFFER_SIZE)
        ; /* non fa niente */

    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;

    // Aggiorna il contatore
    counter++;
}

```

Codice del consumatore:

Il consumatore deve attendere solo se il numero di elementi pieni (`counter`) è zero, ovvero se il buffer è vuoto.

```

while (true) {

    // Attesa se il buffer è vuoto
    while (counter == 0)
        ; /* non fa niente */
}

```

```

    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    // Aggiorna il contatore
    counter--;

    /* consuma l'elemento in next_consumed */
}

```

Race condition:

La logica con il contatore risolve il problema del riempimento di tutte le celle, ma introduce un problema di sincronizzazione (**Race Condition**) sull'accesso alla variabile condivisa `counter`.

Una race condition si verifica quando due o più processi accedono e tentano di modificare dati condivisi concorrentemente, e il risultato finale dipende dall'ordine specifico (e imprevedibile) in cui l'accesso avviene.

IPC in sistemi a scambio di messaggi

È un meccanismo che permette a due o più processi di comunicare e di sincronizzarsi senza condividere lo stesso spazio di indirizzi. È utile negli ambienti distribuiti (macchine diverse su stessa rete).

Il meccanismo di scambio di messaggi richiede almeno due operazioni principali: `send(message)` per inviare un messaggio e `receive(message)` per riceverlo. I messaggi possono avere lunghezza fissa o variabile, con implicazioni diverse per l'implementazione a livello di sistema e per la programmazione dell'applicazione.

Se i processi P e Q vogliono comunicare devono inviare e ricevere messaggi tra loro, dunque deve esistere un canale di comunicazione (*communication link*). Per realizzare a livello logico un canale di comunicazione ci sono vari metodi:

- comunicazione diretta o indiretta;
- comunicazione sincrona o asincrona;
- gestione automatica o esplicita del buffer.

Naming (direct communication)

Ogni processo che vuole comunicare deve nominare esplicitamente il ricevente o il trasmittente della comunicazione. In questo schema, un canale di comunicazione ha le seguenti caratteristiche:

- tra ogni coppia di processi che intendono comunicare si stabilisce automaticamente un canale, i processi devono conoscere solo la reciproca identità;
- un canale è associato esattamente a due processi;
- esiste esattamente un canale tra ciascuna coppia di processi.

Comunicazione indiretta

Con la comunicazione **indiretta** i messaggi vengono inviati a delle porte, o mailbox, che li ricevono. Una mailbox è un oggetto astratto in cui i processi possono introdurre e prelevare messaggi, ed è identificata in modo unico. In questo schema, un processo può comunicare con altri processi tramite un certo numero di mailbox e due processi possono comunicare solo se condividono una mailbox. In questo caso, il canale ha le seguenti caratteristiche:

- tra una coppia di processi si stabilisce un canale solo se entrambi i processi della coppia condividono una stessa mailbox;
- un canale può essere associato a più di due processi;
- tra ogni coppia di processi comunicanti possono esserci più canali diversi, ciascuno corrispondente a una mailbox.

Si supponga che i processi P1, P2 e P3 condividano la stessa mailbox A. P1 invia un messaggio ad A, mentre P2 e P3 eseguono una `receive()` da A. Quale processo riceverà il messaggio dipende dal tipo di schema scelto:

- si fa in modo che un canale sia associato al massimo a due processi;
- si consente l'esecuzione di un'operazione `receive()` a un solo processo alla volta;
- si consente al sistema di decidere arbitrariamente quale processo riceverà il messaggio e successivamente verrà comunicata l'identità del ricevente al trasmittente.

Una mailbox può appartenere:

- ad un processo, ovvero fa parte del suo spazio d'indirizzi. Si distingue tra:
 - il proprietario → può soltanto ricevere messaggi tramite la mailbox

- utente → può solo inviare messaggi alla mailbox

Quando un processo che possiede una mailbox termina, questa scompare.

- al sistema: ha vita autonoma (è indipendente e non è legata ad alcun processo).

Il SO permette a un processo di:

- creare una nuova mailbox;
- inviare e ricevere messaggi tramite la mailbox;
- rimuovere una mailbox.

Sincronizzazione

La comunicazione tra processi avviene attraverso chiamate delle primitive `send()` e `receive()`. Ci sono diversi modi per definire una primitiva, lo scambio di messaggi può essere:

- **Sincrono (o bloccante):**
 - *Invio*: il processo che invia il messaggio si blocca nell'attesa che il processo ricevente, o la mailbox, riceva il messaggio.
 - *Ricezione*: il ricevente si blocca nell'attesa dell'arrivo di un messaggio.
- **Asincrono (o non bloccante):**
 - *Invio*: il processo invia il messaggio e riprende la propria esecuzione.
 - *Ricezione*: il ricevente riceve un messaggio valido o un valore nullo.

Ci sono diverse combinazioni di `send()` e `receive()`, se entrambe sono bloccanti si parla di *rendezvous* tra mittente e ricevente.

Code di messaggi

Se la comunicazione è diretta o indiretta, i messaggi scambiati tra processi comunicanti risiedono in code temporanee. Ci sono tre modi per realizzare queste code:

- **Capacità zero**: la coda ha lunghezza massima 0 $\Rightarrow$ il canale non può avere messaggi in attesa; il trasmittente deve fermarsi finché il ricevente prende in consegna il messaggio. Viene anche chiamato *sistema a scambio di messaggi senza buffering*.
- **Capacità limitata**: la coda ha lunghezza finita $n \Rightarrow$ al suo interno possono risiedere al massimo n messaggi;
 - se la coda non è piena $\rightarrow$ il messaggio viene inserito in fondo alla coda (viene copiato oppure si tiene un puntatore a esso) e il trasmittente continua la propria esecuzione;
 - se la coda è piena $\rightarrow$ il trasmittente deve attendere che ci sia spazio disponibili.
- **Capacità illimitata**: la coda ha una lunghezza potenzialmente infinita; al suo interno può attendere un numero indefinito di messaggi e il trasmittente non si ferma mai.

Gli ultimi due casi vengono anche chiamati *sistemi con buffering automatico*.

Memoria condivisa in posix

Lo standard posix prevede svariati meccanismi per la ipc, tra cui la memoria condivisa: è organizzata utilizzando i file mappati in memoria, che associano la regione di memoria condivisa a un file.

Un processo deve prima creare un oggetto memoria condivisa con la chiamata:

```
fd = shm_open (name, O_CREAT | O_RDWR, 0666);
```

- `name`: nome dell'oggetto memoria condivisa;
- `O_CREAT`: l'oggetto memoria condivisa deve essere creato se non esiste ancora;
- `O_RDWR`: l'oggetto viene aperto in lettura e scrittura;
- `0666`: definisce le autorizzazioni di directory dell'oggetto.

Una volta creato l'oggetto, viene utilizzata la funzione:

```
ftruncate(fd, 4096);
```

per specificare la dimensione dell'oggetto in byte.

Infine, la funzione `mmap()` crea un file mappato in memoria che contiene l'oggetto memoria condivisa e restituisce un puntatore al file mappato in memoria che viene utilizzato per accedere all'oggetto.

Pipe

Una pipe è un canale di comunicazione tra processi. Per essere implementata bisogna tenere in considerazione:

1. La comunicazione permessa:
 - **unidirezionale**
 - **bidirezionale** → può essere:
 - **half-duplex** (i dati viaggiano in unica direzione alla volta);
 - **full-duplex** (i dati viaggiano contemporaneamente in entrambe le direzioni).
2. Se deve esistere una relazione tra i processi in comunicazione (es. padre-figlio).
3. La comunicazione avviene:
 - in **rete**;
 - **sulla stessa macchina**.

Pipe convenzionali

Permettono a due processi di comunicare secondo la modalità produttore-consumatore.

Sono **unidirezionali**, se viene richiesta la comunicazione **bidirezionale** devono essere utilizzate due pipe, ognuna delle quali manda i dati in una direzione.

In Unix sono costruite:

```
pipe(int fd [])
```

Solo il processo che crea la pipe, vi può accedere. Solitamente un processo padre crea una pipe per poter comunicare con il processo figlio.

Inoltre, il processo figlio eredita i file aperti del processo padre e dato che la pipe è un tipo speciale di file, il figlio eredita la pipe dal proprio processo padre.

Named pipe

Quando i processi hanno finito di comunicare, le pipe convenzionali cessano di esistere.

Le named pipe:

- possono essere **bidirezionali**;
- la relazione di parentela non è necessaria.
- diversi processi possono utilizzarla per comunicare;
- continuano a esistere anche dopo che i processi comunicanti sono terminati.

Comunicazione nei sistemi client-server

Socket

Un socket è definito come l'estremità di un canale di comunicazione. Una coppia di processi che comunica attraverso una rete usa una coppia di socket, e ogni socket è identificato da un indirizzo IP concatenato ad un numero di porta.

I socket impiegano l'architettura Client-Server: il **server** attende le richieste dei client, stando in ascolto a una porta specificata; quando il server riceve una richiesta, accetta la connessione proveniente dalla socket del client, e si stabilisce la comunicazione.

I server che svolgono servizi specifici (es. http) stanno in ascolto su porte ben note (es. 80), tutte le porte al di sotto del valore 1024 sono considerate ben note (*well known*) e si usano per realizzare servizi standard.

Tutte le connessioni devono essere uniche, ciò assicura che ciascuna connessione sia identificata da una distinta coppia di socket.

Chiamate di procedura remote

Il paradigma **Remote Procedure Call (RPC)** è un meccanismo che estende il concetto di chiamata di procedura locale permettendo a un programma (client) di eseguire una funzione (procedura) su un sistema remoto connesso tramite una rete, come se la funzione fosse locale.

L'RPC è un tipo di comunicazione tra processi (IPC) ma è specificatamente progettato per sistemi distribuiti e utilizza lo scambio di messaggi come base.

I messaggi RPC sono altamente strutturati:

- **Destinazione:** il messaggio è indirizzato a un demone RPC (un processo server in ascolto) su una porta specifica del sistema remoto.
- **Contenuto:** contiene l'identificatore della funzione da eseguire sul server e i parametri da passarle.
- **Porte:** la porta è un numero logico che, insieme all'indirizzo di rete, identifica il **servizio specifico** sul sistema remoto.

Per mantenere la semplicità della chiamata locale, l'RPC nasconde tutta la complessità della comunicazione di rete tramite gli stub.

1. **Client:** quando il client invoca la procedura remota, il sistema RPC chiama lo stub lato client (di solito c'è uno stub per ogni procedura remota).

2. **Stub client:**

- individua la porta del server;
- esegue la **structuring dei parametri (marshaling)**, convertendo i parametri nel formato di rete standard;
- invia il messaggio RPC al server.

3. **Stub server:**

- riceve il messaggio RPC;
- inverte il *marshaling* (de-strutturazione) dei parametri;
- invoca la procedura effettiva nel server.

4. **Risposta:** se necessario, il server restituisce i risultati al client usando la stessa tecnica.

Il *marshaling* risolve il problema delle differenze di rappresentazione dei dati tra macchine, come ad esempio il problema dell'ordine dei byte (*endianness*).

Molti sistemi RPC per risolvere questo problema, definiscono una **rappresentazione esterna dei dati (XDR)**, un formato di dati indipendente dalla macchina.